

LearnRAG

Technical Documentation — Retrieval-Augmented Generation Pipeline

This document describes the design, components, and data flow of the LearnRAG project: an educational implementation of a Retrieval-Augmented Generation (RAG) system built with LangChain, HuggingFace embeddings, ChromaDB, and Groq-hosted LLM inference.

1. Project Overview

RAG (Retrieval-Augmented Generation) is a technique that improves LLM answers by first retrieving relevant information from an external knowledge source, then providing that information to the model as context before it generates a response. This grounds the model's output in real source material rather than relying purely on what it memorized during training.

LearnRAG implements this pattern end-to-end: it loads a PDF document, splits it into chunks, converts those chunks into vector embeddings, stores them in a vector database, and uses that database to retrieve relevant context for answering user questions with an LLM.

2. Pipeline Architecture

Stage	Module	Description
1. Load	document_loader.py	Reads the PDF and extracts text into LangChain Document objects (one per page).
2. Chunk	chunker.py	Splits documents into smaller, retrievable chunks using recursive or semantic splitting.
3. Embed	embedder.py	Converts each chunk into a dense vector using a local HuggingFace embedding model.
4. Store	vectordb.py	Persists embeddings in a ChromaDB vector database for fast similarity search.
5. Retrieve	vectordb.py	Given a query, finds the top-k most semantically similar chunks.
6. Generate	generator.py	Passes the retrieved chunks as context to a Groq LLM to produce a grounded answer.

Data Flow Diagram

PDF File → document_loader.py → chunker.py → embedder.py → vectordb.py (Chroma) → generator.py (retrieval + Groq LLM) → Final Answer

3. Module Reference

3.1 document_loader.py

Uses LangChain's **PyPDFLoader** to read a PDF file and return a list of Document objects, each representing one page. Each Document has a **page_content** field (the extracted text) and a **metadata** field (source path, page number).

3.2 chunker.py

Splits loaded documents into smaller chunks suitable for embedding. Two strategies are used:

- **Recursive Character Text Splitter** — splits text using a prioritized list of separators (paragraphs, then sentences, then words) to keep chunks structurally coherent. Fast, free, no external API calls. Configurable via `chunk_size` and `chunk_overlap`.
- **Semantic Chunker** — embeds sentences and splits at points where meaning shifts significantly, producing more topically consistent chunks at the cost of extra embedding calls.

3.3 embedder.py

Loads a local HuggingFace embedding model (**sentence-transformers/all-MiniLM-L6-v2**) via **HuggingFaceEmbeddings**. This model converts text into 384-dimension dense vectors and runs entirely on-device, requiring no paid API and no external network calls after the initial model download.

3.4 vectordb.py

Manages a persistent **ChromaDB** vector store built on top of the embeddings from `embedder.py`:

- **build_vector_store()** — embeds a list of chunks and writes them to disk.
- **load_vector_store()** — reloads an existing persisted store without re-embedding.
- **similarity_search()** — given a query string, returns the top-k most similar chunks by vector distance.

3.5 generator.py

Implements the generation stage using LangChain Expression Language (LCEL). The chain:

- Accepts a user question as input.
- Retrieves relevant chunks from the vector store via `similarity_search()`.
- Joins retrieved chunks into a single context string.
- Formats context and question into a prompt template.
- Sends the prompt to a Groq-hosted LLM (llama-3.3-70b-versatile).
- Parses and returns the model's text response.

If the retrieved context does not contain enough information to answer the question, the model is instructed to respond with "I don't know" rather than fabricating an answer.

4. Technology Stack

Component	Tool / Library
PDF Loading	PyPDFLoader (langchain-community)
Text Splitting	RecursiveCharacterTextSplitter, SemanticChunker
Embeddings	HuggingFace sentence-transformers/all-MiniLM-L6-v2 (local)
Vector Database	ChromaDB

LLM Inference	Groq (llama-3.3-70b-versatile)
Orchestration	LangChain Expression Language (LCEL)
Environment Management	python-dotenv

5. Setup Instructions

Install dependencies:

```
pip install -r requirements.txt
```

Create a .env file in the project root with:

```
GROQ_API_KEY=your_groq_api_key_here  
HF_TOKEN=your_huggingface_token_here
```

Run the full pipeline:

```
python src/learnrag/generator.py
```

6. Possible Extensions

- **Hybrid Search** — combine dense retrieval (Chroma) with sparse keyword search (BM25) using LangChain's EnsembleRetriever to improve recall on exact-match queries (IDs, codes, jargon).
- **Evaluation** — build a test set of question/answer pairs to measure retrieval precision and answer faithfulness.
- **UI Layer** — wrap the pipeline in a Streamlit or Gradio interface for interactive use.
- **Multi-document Support** — extend document_loader.py to ingest an entire folder of PDFs instead of a single file.

Document generated as part of the LearnRAG learning project.